SOFTWARE ENGINEERING LAB REPORT

Software Metrics & Complexity Analysis

LAB 10 | Case Study: Food Delivery Application

Project Overview

The Food Delivery Application allows customers to discover restaurants, browse menus, place and pay for orders, track delivery, and provide feedback. Restaurants manage menus and incoming orders, while delivery partners accept assignments and update delivery status. An administrator supervises users, restaurants, payments, and operational issues.

Submitted by

Team Member	Roll Number
Deepak Singh	202401100200123
Deepak Maurya	202401100200121
Deepak Nimesh	202401100100122
Deependra Ahirwar	202401100200124

Department of Computer Science & Engineering

1. Aim

To identify useful software metrics and estimate algorithmic complexity for important Food Delivery Application operations.

2. Software Metrics

Metric	Definition / Use	Example Target or Interpretation
Lines of Code (LOC)	Approximate implementation size; not a direct quality measure.	Track trends by module; avoid treating more LOC as better.
Cyclomatic Complexity	V(G) = E – N + 2P; estimates independent paths in control flow.	Keep critical functions simple; review functions with high complexity.
Code Coverage	Executed statements/branches divided by total measured statements/branches.	Aim for strong coverage on payment, pricing, and order-state logic.
Defect Density	Confirmed defects per size unit, often per KLOC.	Compare modules and releases consistently.
Response Time	Elapsed time from request to response; track median and tail percentiles.	Track p50, p95, and p99 for menu and checkout APIs.
Availability	Uptime divided by total observed time.	Define a service objective appropriate to project scale.
MTTR	Mean time to restore service after an incident.	Lower MTTR indicates faster recovery.
Change Failure Rate	Share of deployments causing incidents or rollback.	Use for release-process improvement.

3. Complexity Analysis of Core Operations

Operation	Approach	Time Complexity	Notes
Search restaurants	Linear scan without index	O(R)	An index on location/category can reduce candidate search cost.
Search menu items	Scan items for a restaurant	O(M)	A database index or search index improves lookup.
Calculate cart total	Sum prices of C cart lines	O(C)	Includes quantity and discount/tax calculations.
Create order	Validate cart and persist C order lines	O(C)	Database transaction and indexes affect real latency.
Sort restaurants by rating	Comparison sort of R results	O(R log R)	Can be pushed to database with a suitable index when appropriate.
Assign nearest rider	Compute distance for D available riders	O(D)	Spatial indexes or geospatial queries can reduce candidate set.
Order history pagination	Indexed query returning K rows	Typically O(log N + K)	Depends on index, pagination strategy, and query plan.
Notification fan-out	Send to A recipients	O(A)	Use a queue and bounded concurrency for large fan-out.

4. Example Cyclomatic Complexity

Consider a checkout function with decisions for an empty cart, valid address, available items, and successful payment. If the control-flow graph has E = 12 edges, N = 9 nodes, and P = 1 connected component, then V(G) = E – N + 2P = 12 – 9 + 2 = 5. This indicates five linearly independent paths that should inform test design. The value is

illustrative; actual complexity must be calculated from implemented code.

5. Suggested Test Paths

• Empty cart → reject checkout.

• Non-empty cart + invalid address → show validation error.

• Valid cart + unavailable item → request cart correction.

• Valid checkout + payment failure → do not confirm the order.

• Valid checkout + payment success → create/confirm order and trigger restaurant workflow.

6. Quality Measurement Plan

Area	How to Measure	Frequency
Correctness	Automated unit/integration test pass rate and escaped defects	Every change/release
Performance	p50/p95/p99 API latency, throughput, error rate	Load tests and production-like runs
Reliability	Availability, failed jobs, payment reconciliation mismatches	Continuous monitoring
Maintainability	Complexity hotspots, duplicated code, review findings	Each sprint
Security	Dependency scan, access-control tests, secret scan	Each release

7. Result and Conclusion

The selected metrics cover code complexity, testing, performance, reliability, and maintainability. Complexity estimates are useful for planning tests and identifying hotspots, but should be interpreted with real profiling, code review, and operational evidence rather than used as standalone quality scores.